Slide 01 | Decentralized Role-Leasing IoT Protocol



Speaking guide

Introduce this as a software simulation of coordination in agricultural IoT. The central contribution is selecting temporary role holders using recursive capability, migration costs, and lease rules.

The supplied folder does not identify the student, institution, department, or guide, so replace the marked fields before presenting. The implementation and stored experiments support statements about this model, not a deployed farm network. Allow about 18 minutes for slides 1–33 and use the appendix for questions.

Implementation evidence: Project.md; simulation/config.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 02 | Project overview

Project overview

01 Python simulation of a fixed logical hierarchy over a physical multi-hop mesh

02 100 nodes, 10 subnets, 3 networks, 1 internetwork by default

03 Four role labels at 14 scopes produce 56 required assignments

04 Users: researchers and students comparing coordination policies

simulation/config.py; simulation/entities.py; nodes.py

02 / 33

Speaking guide

Explain the distinction between a role and a scope. A gateway role exists separately for each subnet, each network, and the internetwork. Candidates are immediate children: nodes within a subnet, subnets within a network, and networks at the root.

The system measures service availability and communication costs under changing conditions. Anomaly detection, health monitoring, and discoverability are role labels and scores. The code does not implement their application payload services. Transition to why reassignment is useful.

Implementation evidence: simulation/config.py; simulation/entities.py; nodes.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 03 | Problem statement

Problem statement

01 Fixed role holders may remain assigned after their energy or connectivity deteriorates

02 Global coordination creates a dependency on a controller in the modeled baselines

03 Frequent reassignment consumes communication energy and can cause instability

04 The engineering problem is balancing capability improvement against switching cost

Project.md; simulation/protocols.py; simulation/runner.py

03 / 33

Speaking guide

Present this as the problem investigated by the project rather than a claim about every IoT deployment. Node energy and link quality affect both suitability and communication. Choosing the highest current score at every step can produce unnecessary churn.

Keeping every original holder can leave roles unavailable. The simulator creates a controlled way to compare those policies on the same initial world and event schedule. Ask the panel to judge the mechanism and the evidence under the stated abstraction.

Implementation evidence: Project.md; simulation/protocols.py; simulation/runner.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 04 | Motivation and objectives

Motivation and objectives

01 Preserve useful role availability when nodes fail or conditions change

02 Reuse the same role semantics across hierarchy levels

03 Account for routed control traffic, data traffic, and endpoint energy

04 Make experiments repeatable with seeded inputs and replayable events

Project.md; FINAL_SPRINT_PLAN.md; simulation/events.py

04 / 33

Speaking guide

The specification motivates agricultural networks with distributed, potentially solar-powered devices. In implementation, this becomes a deterministic experiment framework. Its objectives are observable: capability refresh, temporary assignments, failure recovery, and separate communication counters.

Solar harvesting is a later implemented extension. Do not suggest that the project contains real agricultural sensor readings or hardware measurements. The hierarchy is an input assumption, so automatic clustering is outside the current objectives.

Implementation evidence: Project.md; FINAL_SPRINT_PLAN.md; simulation/events.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 05 | Implemented comparison policies

Implemented comparison policies

Policy	Control source	Holder update
centralized	Fixed controller	Best capability, immediate
hierarchical	Random fixed controller	Random initial / failure replacement
static	Scope representative	Initial assignment only
dynamic	Scope representative	Best capability, immediate
proposed	Scope representative	Cost, hysteresis, lease lifecycle

simulation/protocols.py; simulation/runner.py

05 / 33

Speaking guide

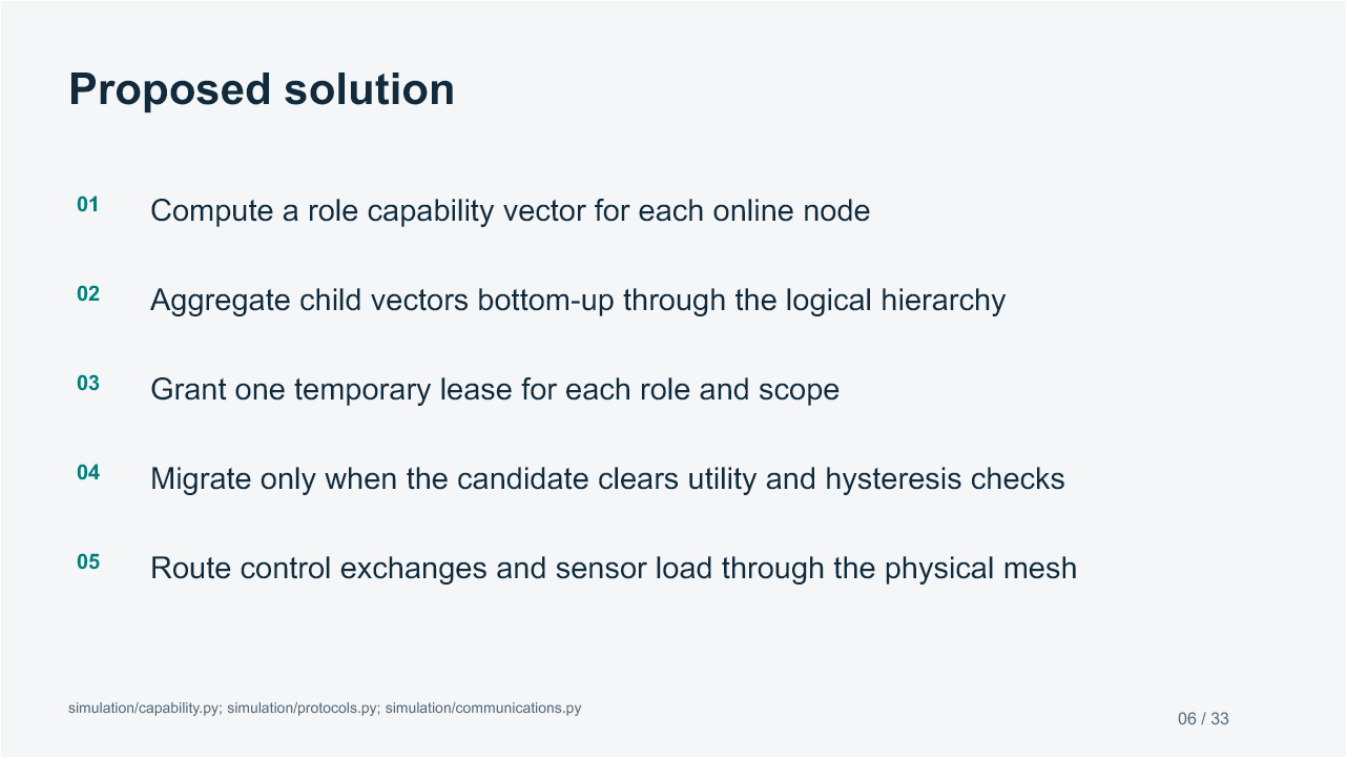
Read each row as the exact policy in this codebase. The hierarchical baseline chooses a seeded random fixed controller and random reachable eligible holders, retaining them until failure. Centralized instead uses a fixed controller selected by normalized degree and capability-based reassignment.

Static stops policy updates after initialization. Dynamic has local control sources and immediately selects the best reachable score. Proposed adds cost-aware optional migration and leases. These are custom model baselines, not implementations of LEACH, RPL, or another named networking standard.

Implementation evidence: simulation/protocols.py; simulation/runner.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 06 | Proposed solution



Speaking guide

Describe the mechanism as several cooperating modules. The capability calculation says who is suitable. The protocol engine decides whether a change is justified.

The lease state machine records who currently holds a role. The router makes communication success and energy visible. The system is decentralized in modeled control-source selection, but one Mesa model executes the simulation in a single process. It does not implement distributed consensus or independent devices exchanging real packets.

Implementation evidence: simulation/capability.py; simulation/protocols.py; simulation/communications.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 07 | Technology stack and tradeoffs

Technology stack and tradeoffs

Component	Actual technology	Reason / tradeoff
Runtime and agents	Python 3 + Mesa	Inspectability and repeatable model clock
State transitions	transitions	Explicit legal node and lease states
Numerics and records	NumPy + Pandas	Seeded worlds and tabular exports
Routing	dict + heapq	Transparent custom Dijkstra logic
Reporting	Matplotlib + SciPy	Plots and paired t intervals

requirements.txt; nodes.py; simulation/*.py; patent-study/analyze_patent_study.py

07 / 33

Speaking guide

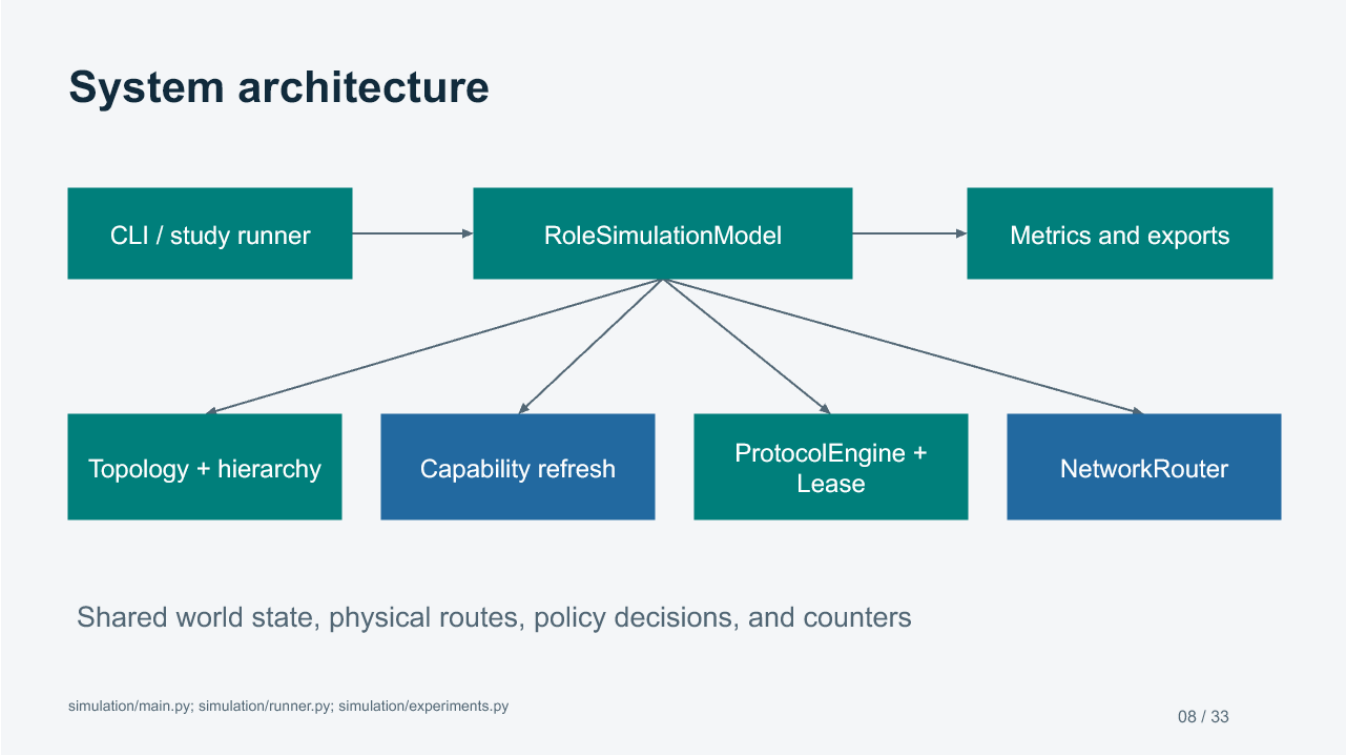
These responsibilities are visible in imports and code. Mesa supplies agent identity and a model clock, while transitions enforces legal state changes. NumPy supplies seeded random generators and numeric operations. Pandas exports and aggregates metrics. Matplotlib produces plots. SciPy is used for t intervals by the study analyzer, but requirements.txt does not declare it.

The original specification mentions SimPy and NetworkX, but the active simulator uses neither. Adjacency dictionaries and a custom heap-based router implement the graph behavior. These rationales are architectural interpretations, not recorded author interviews. Environment verification found that Mesa 3.5.1 conflicts with assigning a string to model.scenario. Short simulations work with Mesa 3.3.1. Pin and test a compatible Mesa version before the live demo. NetworkX is a Mesa dependency in this environment, but this project does not use it directly for topology or routing.

Implementation evidence: requirements.txt; nodes.py; simulation/*.py; patent-study/analyze_patent_study.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 08 | System architecture



Speaking guide

Follow the diagram from the CLI or study runner into RoleSimulationModel. Configuration and the event schedule define the world. Topology creates positions and the physical adjacency map. EntityHierarchy maps the logical organization.

Capability refresh feeds ProtocolEngine, which owns leases and uses NetworkRouter for grants, renewals, and migration. Sensor traffic uses that same router. MetricsRecorder reads the resulting state and counters. The experiment layer exports CSV and PNG, while the study layer additionally writes compressed NumPy time series. The diagram represents runtime dependencies, not separate servers.

Implementation evidence: simulation/main.py; simulation/runner.py; simulation/experiments.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 09 | Project structure

Project structure

Path	Responsibility
nodes.py	Node attributes, capability, online / failed state
simulation/topology.py + entities.py	Mesh construction and hierarchy mapping
capability.py + leases.py + protocols.py	Aggregation, lifecycle, selection, migration
communications.py + runner.py	Routing, energy, tick coordination
events.py + metrics.py + experiments.py	Replay, measurement, exports
tests/ + patent-study/	Regression suite and matched study

simulation/; nodes.py; placer.py; FINAL_SPRINT_PLAN.md

09 / 33

Speaking guide

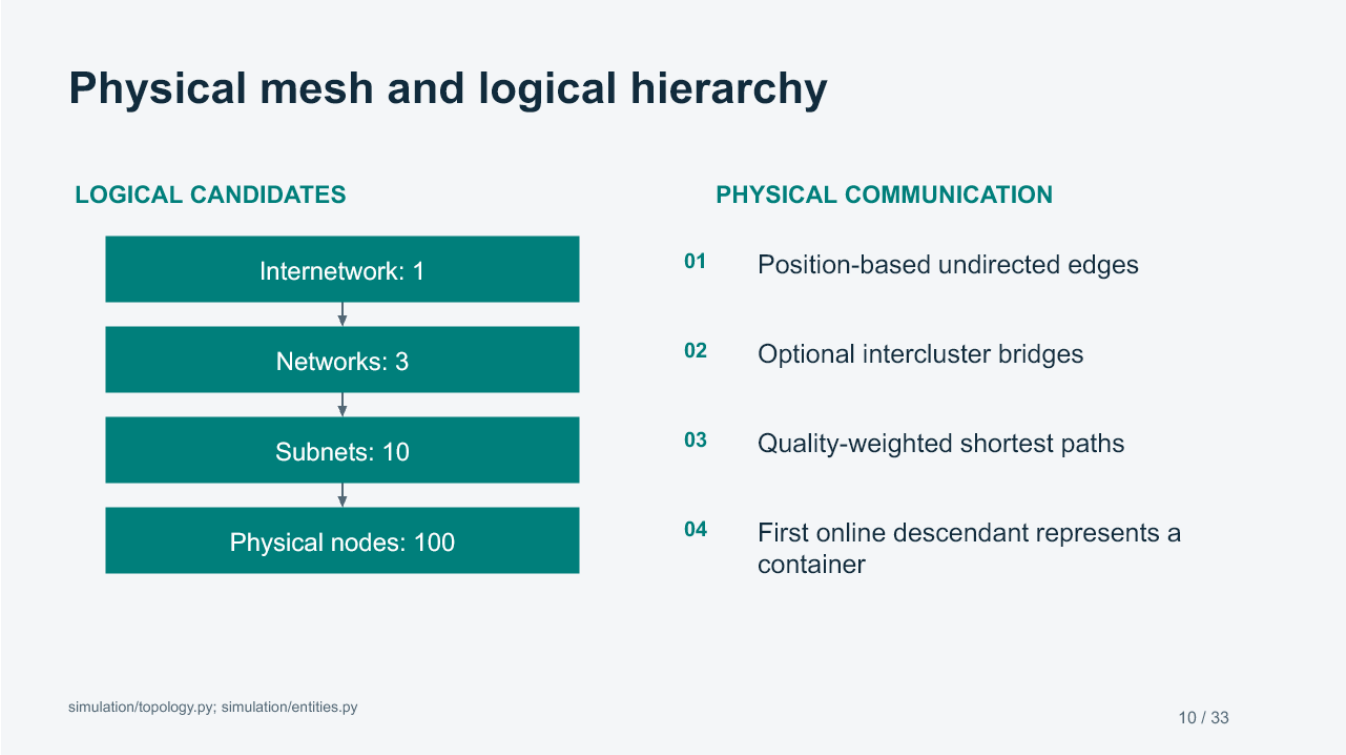
The active implementation is the simulation package plus nodes.py. The standalone placer.py creates a map and visualization but the active runner directly calls create_topology rather than loading network_map.csv. The patent-study folder contains execution, aggregation, and stored evidence.

Older HANDOFF.md and Roadmap.md describe a hierarchy.py monolith that is absent from this folder. FINAL_SPRINT_PLAN.md explicitly supersedes that direction. Use the current package names when explaining code and avoid repeating old lease counts or incomplete status from the handoff.

Implementation evidence: simulation/; nodes.py; placer.py; FINAL_SPRINT_PLAN.md

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 10 | Physical mesh and logical hierarchy



Speaking guide

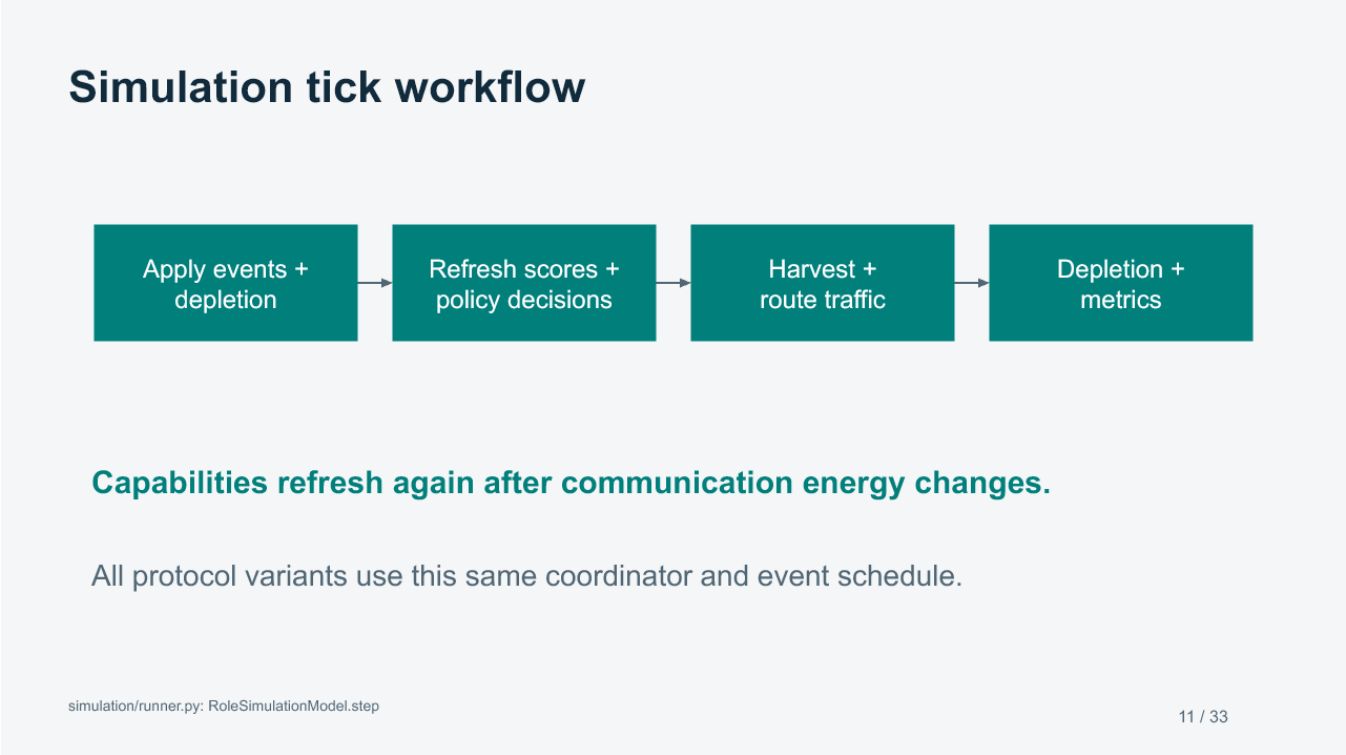
Read the left chain as candidate membership, not radio links. The default subnet distribution is 4, 3, and 3. A container is available if at least one descendant node is online. Its representative is the first available descendant in stored order, not necessarily its best gateway.

On the physical side, distance creates undirected edges and normalized degree supplies centrality. Positive intercluster spacing also inserts nearest bridge links between adjacent clusters. These long-range links are an abstraction and can exceed the nominal communication range. No hierarchy relationship automatically provides a route.

Implementation evidence: simulation/topology.py; simulation/entities.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 11 | Simulation tick workflow



Speaking guide

Mesa advances the step count and invokes the model step. Apply scheduled events first and fail depleted nodes. Refresh capabilities before policy decisions so failure and degradation influence selection.

Mark affected scopes if configured, then update leases and migrations. Harvest solar energy after protocol decisions and before sensor traffic. Route sensor and optional backhaul traffic, fail newly depleted nodes, refresh capabilities again, and record metrics. This ordering matters: a node recharged during harvesting can rejoin, while a missing lease may only be assigned at a later policy update.

Implementation evidence: simulation/runner.py: RoleSimulationModel.step

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 12 | Node capability scoring

Node capability scoring

01 Gateway = 0.40E + 0.35Q + 0.25C

02 Anomaly detection = health monitoring = 0.50E + 0.50H

03 Discoverability = 0.34E + 0.33Q + 0.33H

04 Example: E=.9, H=.8, Q=.7, C=.6 gives gateway .755

05 Failed nodes contribute zero capability

nodes.py: ROLE_WEIGHTS; NodeAgent.capability

12 / 33

Speaking guide

Define E as normalized battery energy, H as health, Q as node link quality, and C as normalized initial degree. ROLE_WEIGHTS in nodes.py is the single source of truth. Gateway capability in the example is .36 plus .245 plus .15, which is .755. Both anomaly detection and health monitoring produce .85 because their weight definitions are identical.

Discoverability produces .801. Scores are bounded to [0,1]. A positive capability does not guarantee a route, which is why selection also checks reachability. No learned model or classifier appears here.

Implementation evidence: nodes.py: ROLE_WEIGHTS; NodeAgent.capability

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 13 | Recursive capability aggregation

Recursive capability aggregation

01 weighted_aggregate() uses normalized non-negative child weights

02 Missing child weights default to 1.0, so the default is a mean

03 Subnet vectors feed network vectors, then the internetwork vector

04 Failed children remain in the denominator with zero capability

05 Unequal child populations make a mean of means differ from a global node mean

simulation/capability.py: weighted_aggregate; refresh_capabilities

13 / 33

Speaking guide

For child gateway scores .8 and .6 with weights 1 and 3, the aggregate is (.8 + 1.8) / 4 = .65. Empty children return a zero vector. Invalid negative weights or a nonpositive total raise an error.

Refresh starts with all leaf vectors and updates each parent in dependency order. The default root gives equal weight to networks despite their 4/3/3 subnet counts. That is a hierarchy weighting choice, not a population-weighted global average. Recomputing from children keeps the function deterministic and independent of previous aggregate values.

Implementation evidence: simulation/capability.py: weighted_aggregate; refresh_capabilities

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 14 | Selection and assignment

Selection and assignment

01 Candidates are immediate children of the requested scope

02 Filter unavailable entities and unreachable representatives

03 Choose maximum capability with a stable string-ID tie break

04 Send LEASE_GRANT before creating the active lease

05 Store leases in dict[(role, scope)] to enforce one holder per pair

simulation/protocols.py: _select_for_role; _assign

14 / 33

Speaking guide

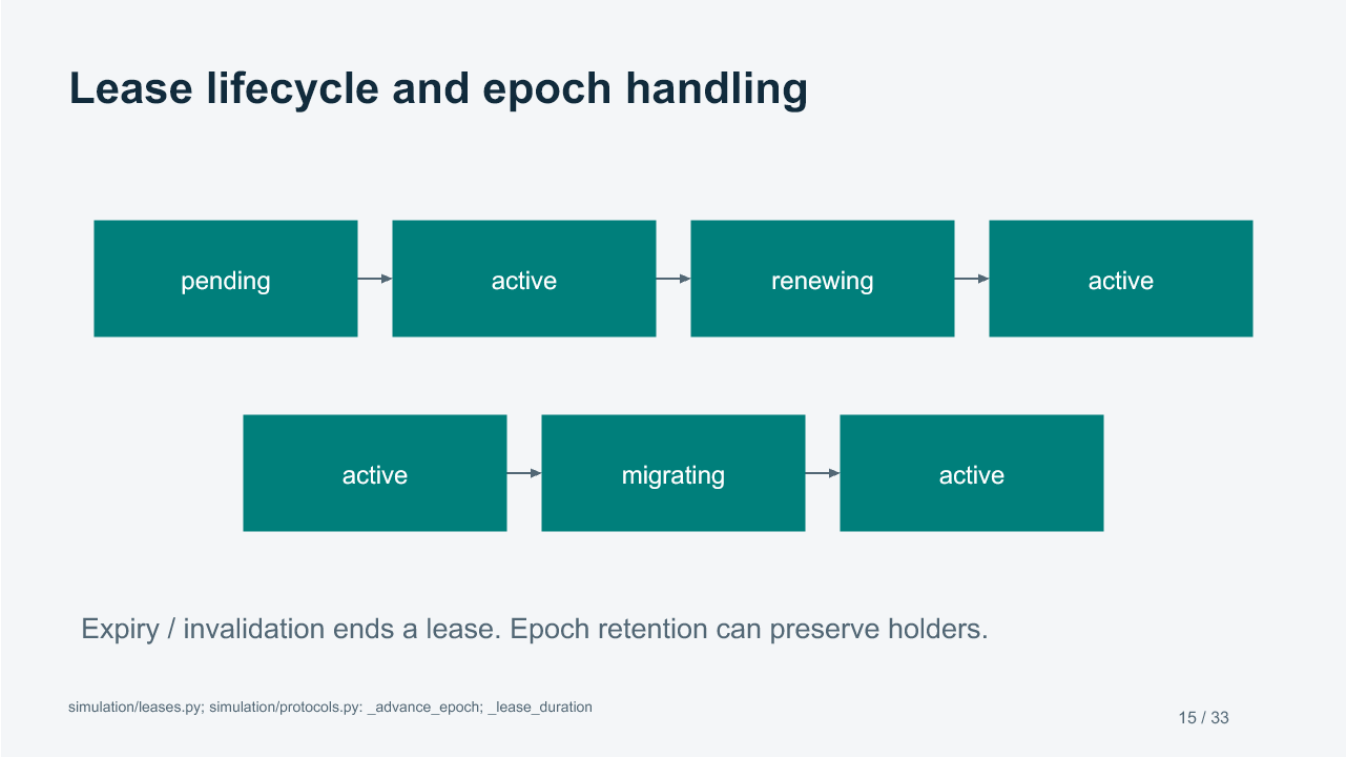
Explain _select_for_role and _assign together. A scope needs a control source and at least one reachable candidate. The proposed initial assignment is capability-based and does not deduct migration cost because no holder is being replaced. The hierarchical policy instead uses seeded random choice.

A failed grant leaves the role unassigned. A successful grant creates a Lease and activates it. The tuple-keyed dictionary gives direct access to each role/scope assignment. Multiple roles may share a holder because the code has no capacity limit or exclusivity across different role keys.

Implementation evidence: simulation/protocols.py: _select_for_role; _assign

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 15 | Lease lifecycle and epoch handling



Speaking guide

The transitions library prohibits illegal jumps. New leases start pending and activate. Renewal temporarily enters renewing and updates timing before returning active. Migration enters migrating and replaces holder, epoch, and timing in the same lease object.

Expiry or invalidation ends availability. Duration in proposed is 10 + capability times 90 by default, so a .8 score gives 82 ticks and renewal becomes due at 57.4 ticks, checked on integer ticks. Epochs advance every 20 ticks. Default behavior regrants assignments, while retention updates existing epoch labels in place. This is an in-process fencing model, not a distributed stale-message rejection protocol.

Implementation evidence: simulation/leases.py; simulation/protocols.py: _advance_epoch; _lease_duration

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 16 | Cost-aware migration

Cost-aware migration

01 Cost = α·2 + β·state_transfer + γ·route_cost·Econtrol + δ·disruption

02 Defaults: α=.01, β=.1, γ=10, δ=.2, λ=.1

03 Candidate utility = capability − λ·cost

04 Migrate only if best utility > current capability + Δ

05 Unreachable transfer routes have infinite cost and are excluded

simulation/protocols.py: calculate_migration_cost; _consider_cost_aware_migration

16 / 33

Speaking guide

The cost terms are deterministic estimates. The control-message term is fixed at two, while _migrate sends one modeled MIGRATE exchange from the scope source to the new holder. Thus the utility estimate and actual charged control energy are related but distinct models. With route cost 6 and default control energy .00001, cost is .020 + .025 + .0006 + .050 = .0956.

Candidate score .88 gives utility .87044. Against current .80 and delta .05, migration passes. A .85 candidate gives .84044 and fails. Optional cooldown and adaptive hysteresis can further restrict changes.

Implementation evidence: simulation/protocols.py: calculate_migration_cost; _consider_cost_aware_migration

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 17 | Routing algorithm and example

Routing algorithm and example

01 Dijkstra minimizes sum of edge costs using a min-heap

02 Usable quality = base edge quality × min(endpoint link qualities)

03 Edge cost = 1 + 4 × (1 − quality)

04 Example: direct quality .2 costs 4.2, two quality .9 hops cost 2.8

05 Cache routes by source and invalidate after topology-affecting events

simulation/communications.py: _routes_from; _current_graph; find_route

17 / 33

Speaking guide

This algorithm prefers the two-hop route in the example even though it uses more hops. A link at or below minimum_link_quality is excluded. Failed nodes lose incident usable links. Sorted neighbors and a numerical tolerance stabilize equal-cost behavior.

Source equals destination returns a zero-hop successful route. A missing path returns None and increments a delivery-failure counter when a message is offered. Cache invalidation happens on failure, recovery, link degradation, and depletion. The router stores explicit path tuples for all reachable destinations, so path materialization adds overhead beyond the textbook Dijkstra bound.

Implementation evidence: simulation/communications.py: _routes_from; _current_graph; find_route

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 18 | Communication and energy accounting

Communication and energy accounting

01 Count logical messages separately from message-hop transmissions

02 Each hop charges both transmitter and receiver

03 Transmit = Ebase × [1 + w × (distance / range)^p]

04 Receive = Ebase, with data and control using different base costs

05 Failed routing counts an offered item, while no route means no hop energy

simulation/communications.py: send; _hop_energy_cost; _record_energy; nodes.py

18 / 33

Speaking guide

Default data energy is .00005 per endpoint and control energy .00001. Distance weight defaults to zero, while the stored clustered study uses weight one and exponent two. A zero-hop message succeeds without communication energy.

Accounting records requested modeled costs even when battery depletion clamps the actual battery at zero, so communication energy and residual battery loss are not interchangeable conservation totals. Explicit degradation and harvested energy also change batteries. The model omits retransmissions, interference, sensing, CPU work, and idle consumption. All energy values use normalized units rather than joules.

Implementation evidence: simulation/communications.py: send; _hop_energy_cost; _record_energy; nodes.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 19 | Solar harvesting and migration gate

Solar harvesting and migration gate

01 Seeded node harvest = configured rate × uniform(.5, 1.5)

02 Battery capacity caps accepted harvest and records curtailed energy

03 Energy-depleted nodes rejoin after reaching the reserve threshold

04 Optional gate compares projected reserve benefit with transfer energy + margin

05 Failure replacement bypasses the optional migration gate

simulation/runner.py: _harvest_solar_energy; simulation/protocols.py: _solar_migration_is_worthwhile

19 / 33

Speaking guide

Each node has a constant harvest rate for the entire run, with a separate seeded generator to avoid changing topology randomness. The projection uses battery plus horizon times harvest minus observed average communication use, then clamps capacity and subtracts essential reserve. Default horizon is 500 steps and margin is .01.

Container reserve projections average available descendants. The gate does not replace capability and migration-cost ranking. It acts after hysteresis and applies only to optional migrations. This is a simple extrapolation without daylight or weather cycles, and the stored results must determine whether it actually helps.

Implementation evidence: simulation/runner.py: _harvest_solar_energy; simulation/protocols.py: _solar_migration_is_worthwhile

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 20 | Data flow and persistence

Data flow and persistence



Sensor delivery measures the subnet leg.

Optional backhaul sends aggregates. Counters become CSV, PNG, and NPZ files.

simulation/runner.py: _route_sensor_traffic; simulation/metrics.py; simulation/experiments.py

20 / 33

Speaking guide

Distinguish sensor-item delivery from hierarchical backhaul. Every currently online node offers one sensor item to its subnet gateway. The sensor delivery metric counts that first-leg success.

When backhaul is enabled, successful subnets produce one aggregate to a network gateway, and active networks produce one aggregate to the internetwork gateway. Those aggregate deliveries have separate counters and do not redefine sensor-item delivery as end-to-end root receipt. Metrics accumulate in memory and export to CSV, PNG, and study NPZ files. There is no database, web API, or persistent server in this project.

Implementation evidence: simulation/runner.py: _route_sensor_traffic; simulation/metrics.py; simulation/experiments.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 21 | Internal interfaces and failure handling

Internal interfaces and failure handling

Interface	Input	Output / failure
RoleSimulationModel(config, protocol, schedule)	Validated world and events	One simulation world / ValueError
run_steps()	Optional bounded step count	State and metric records
find_route(source, destination)	Physical integer node IDs	RouteResult or None
ProtocolEngine.assignments	(role, scope) key	Lease object or absent key
MetricsRecorder.summary(scenario, seed)	Run labels	Metric dictionary

simulation/config.py; simulation/events.py; simulation/runner.py; simulation/communications.py

21 / 33

Speaking guide

These are Python calls in one process, not HTTP endpoints. Configuration validates finite values, dimensions, ranges, and lease settings. Event construction rejects unsupported kinds and invalid degradation values. Model initialization rejects unknown node references and events beyond the configured horizon.

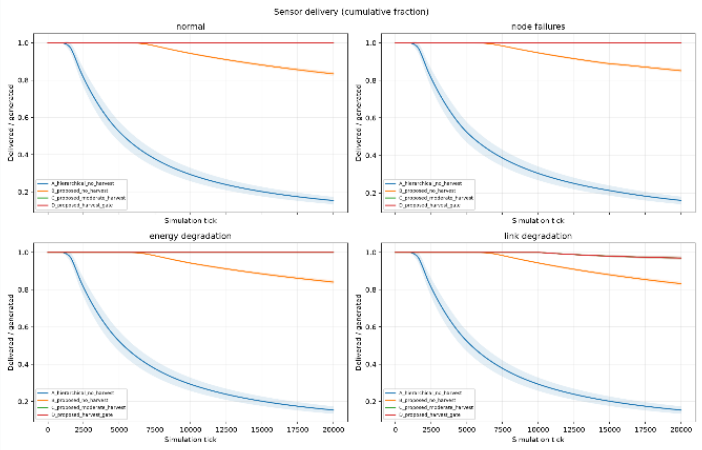
Routing rejects invalid endpoint IDs but returns None for unavailable or disconnected endpoints. Lease transitions reject invalid lifecycle operations. Security is currently about input validation and state consistency. Authentication, authorization, encryption, and protection from malicious IoT nodes are absent because no actual network service is exposed.

Implementation evidence: simulation/config.py; simulation/events.py; simulation/runner.py; simulation/communications.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 22 | Actual application outputs

Actual application outputs



Genuine saved output. No graphical application UI is implemented.

patent-study/results_20000/sensor_delivery.png; simulation/main.py

22 / 33

Speaking guide

Show the genuine stored figure on this slide as an application output, not a product UI screenshot. Its panels come from patent-study/results_20000/sensor_delivery.png. The study compares A hierarchical without harvest, B proposed without harvest, C proposed with moderate harvest, and D proposed with harvest plus the gate.

These labels differ from the five generic CLI policies. Do not invent a dashboard. For a live demonstration, use the next slide and open the generated summary and migration CSV. The plot represents simulated cumulative sensor delivery under the documented study setup.

Implementation evidence: patent-study/results_20000/sensor_delivery.png; simulation/main.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 23 | Demonstration workflow

Demonstration workflow

01 1 Run a short matched experiment with seed 42

02 2 Compare proposed and hierarchical under node_failures

03 3 Inspect summary.csv for delivery and role continuity

04 4 Inspect migration and time-series CSVs around failure / recovery

05 5 Use the full study only when explaining long-horizon findings

simulation/main.py; simulation/events.py; simulation/experiments.py

23 / 33

Speaking guide

Suggested command: python -m simulation.main --seed 42 --steps 100 --scenarios node_failures --protocols hierarchical proposed --output demo_results. With the default 100-node event generator, ten selected nodes fail at tick 25 and recover at tick 75. Failures may include nonholders, and a random baseline controller failure has an especially large effect.

Open summary.csv, then timeseries/node_failures_proposed_timeseries.csv and migrations/node_failures_proposed_migrations.csv. A small run verifies behavior, while the stored 20,000-step experiment addresses longer horizons. Current verification status and actual smoke results appear on the testing slide.

Implementation evidence: simulation/main.py; simulation/events.py; simulation/experiments.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 24 | Technical challenges and solutions

Technical challenges and solutions

Challenge	Implementation response
Score definitions drift across modules	Single ROLE_WEIGHTS table
Logical hierarchy confused with physical links	Separate membership and adjacency maps
Message counts hide routed overhead	Separate logical, hop, and energy counters
Epoch revocation masks lease behavior	Optional valid-lease retention
Availability metrics hide missing items	Explicit generated / delivered denominators

Roadmap.md; finished_work.md; simulation/metrics.py; simulation/protocols.py

24 / 33

Speaking guide

These challenges are supported by code and development records rather than invented personal anecdotes. Early duplicated role weights motivated the single definition in nodes.py. Separating mesh from hierarchy prevents membership from silently creating radio links.

The router separates logical messages from physical hop counts. The development record notes that 20-tick epoch revocation can mask longer lease renewal and batching behavior, leading to configurable retention. Explicit generated-item counts fix denominator ambiguity. A remaining concern is that shared memory gives immediate global capability knowledge unless periodic broadcasts are configured.

Implementation evidence: Roadmap.md; finished_work.md; simulation/metrics.py; simulation/protocols.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 25 | Testing and validation

Testing and validation

82 passed

Fresh regression execution during this review

01 480 stored run records and 30 time-series archives checked

02 Two fresh 100-step node-failure simulations completed

03 No coverage percentage or hardware benchmark claimed

test_nodes.py; tests/test_*.py

25 / 33

Speaking guide

Explain testing by invariant rather than only by count. Arithmetic tests check role weights and normalized aggregation. Router tests verify reliable multihop selection, same-node success, partitions, message counts, and distance energy. Protocol tests cover one assignment per key, hysteresis, renewal at the configured fraction, epoch retention, broadcasts, and recovery delay.

Solar tests cover capacity, reproducible rates, reserve recovery, and gate acceptance/rejection. No coverage percentage or runtime benchmark is available. The original history reports 80 tests, while the fresh run is the relevant present result. Fresh test execution: 82 passed in 9.47s. Smoke results: hierarchical: CSC 1.0000, sensor delivery 1.0000; proposed: CSC 1.0000, sensor delivery 1.0000.

Implementation evidence: test_nodes.py; tests/test_*.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 26 | Stored experimental design

Stored experimental design

Arm	Protocol	Harvest	Gate
A	hierarchical	0	off
B	proposed	0	off
C	proposed	.0002 × node factor	off
D	proposed	.0002 × node factor	on

patent-study/run_patent_study.py; patent-study/results_20000/final_report.md

26 / 33

Speaking guide

The supplied report declares 480 completed runs: 30 seeds times four scenarios times four arms. Seeds span 42–71 and each run lasts 20,000 ticks. Arms share the initial topology and event schedule within each seed and scenario.

The study uses 100 nodes, intercluster gap 1.5 communication ranges, distance energy weight one, hierarchical backhaul, and epoch retention. Moderate harvest is .0002 before the seeded per-node multiplier. Because A and B also differ in controller and selection behavior, B minus A measures a bundled protocol contrast. C minus B isolates harvest within proposed, and D minus C isolates the optional gate.

Implementation evidence: patent-study/run_patent_study.py; patent-study/results_20000/final_report.md

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 27 | Metric definitions and interpretation

Metric definitions and interpretation

01 CSC = available role-time intervals / required role-time intervals

02 Default denominator = 56 roles × elapsed ticks, excluding time zero

03 Sensor delivery = delivered sensor items / generated sensor items

04 Energy per item = modeled communication energy / delivered sensor items

05 Lifetime records first threshold crossing and right-censors absent crossings

simulation/metrics.py; patent-study/analyze_patent_study.py

27 / 33

Speaking guide

CSC is a role-state availability proxy. It does not prove that anomaly detection or another application service performed useful computation. Sensor delivery includes unrouted generated items in the denominator, but only online nodes generate traffic, so compare offered load as well as fractions.

It also measures the subnet leg rather than root delivery. The stored analyzer uses paired seed differences and 95 percent t intervals. Service lifetime uses cumulative fractions at 90 and 50 percent thresholds, while network lifetime uses instantaneous active-node fraction below 50 percent. Censored horizon values are not observed failure times.

Implementation evidence: simulation/metrics.py; patent-study/analyze_patent_study.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 28 | Verified stored results: normal scenario

Verified stored results: normal scenario

Arm	CSC	Sensor delivery	Energy/item $\times 10^{-6}$	Total energy	Final battery
A	26.113%	15.37%	171.38	40.80	0.467
B	99.998%	83.38%	117.37	85.16	0.023
C	100.000%	100.00%	154.38	297.08	0.822
D	100.000%	100.00%	154.82	297.81	0.819

30 seeds per arm. Normalized energy units. Means of per-run ratios.

patent-study/results_20000/per_run_metrics.csv

28 / 33

Speaking guide

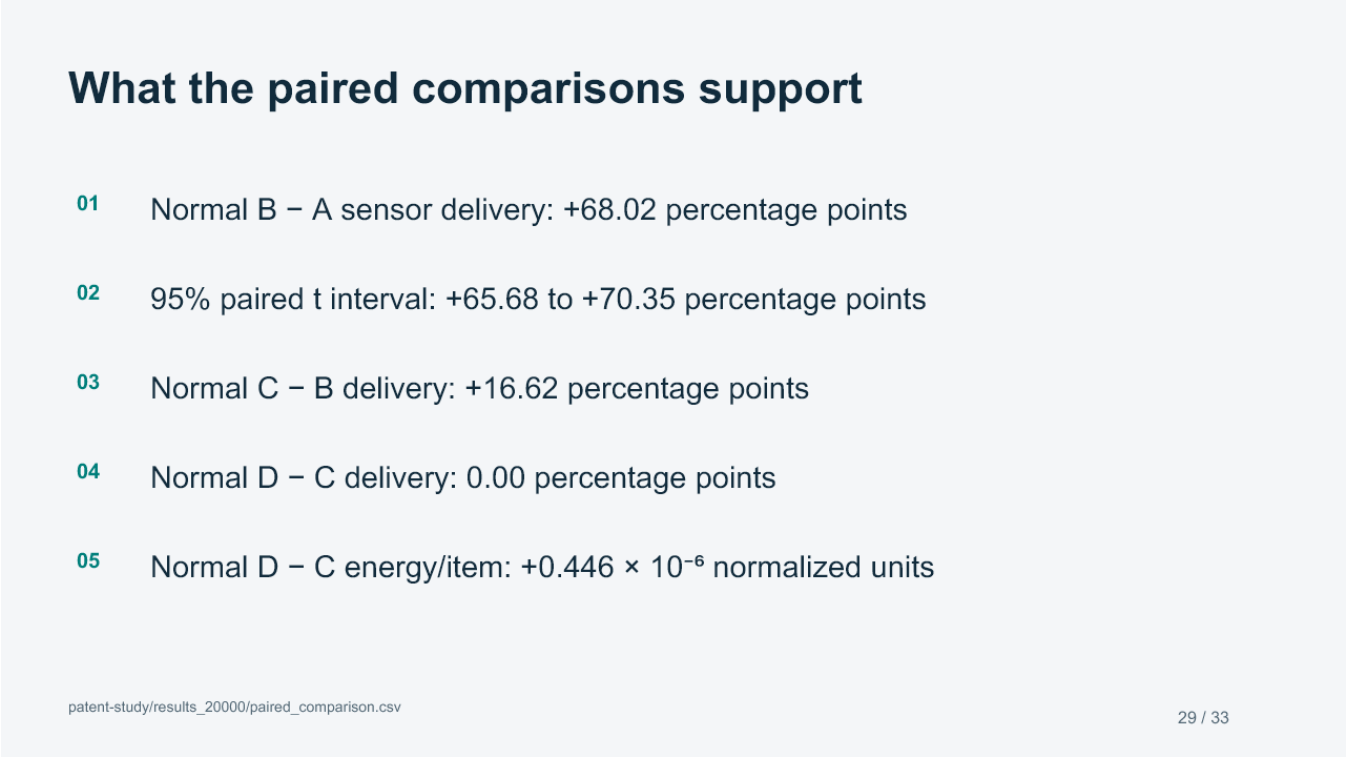
These are per-run means recomputed from the supplied CSV, rounded for display. A retains substantial battery but loses much of the modeled service because coordination can fail. B has much higher role continuity and sensor delivery but low ending battery.

C and D reach one in the stored sensor and continuity metrics under this configuration, with higher total communication energy because they sustain service longer. This is not evidence of perfect radio delivery in the field. Fractions are means of run fractions, and energy per item is the mean of per-run ratios, so it need not equal the ratio of displayed means.

Implementation evidence: patent-study/results_20000/per_run_metrics.csv

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 29 | What the paired comparisons support



Speaking guide

Use percentage points for fraction differences. The stored normal B minus A delivery effect is .6801577 with interval [.6568311, .7034843]. C minus B is .1661842 with interval [.1554166, .1769518]. D minus C has zero delivery difference in normal, failures, and energy degradation.

In link degradation its delivery interval includes zero. In normal, D slightly increases energy per delivered item, with a positive interval in the stored analysis. Therefore the gate does not establish a general improvement over harvest alone. These intervals describe this paired simulation design and do not establish causality for real devices or universal superiority.

Implementation evidence: patent-study/results_20000/paired_comparison.csv

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 30 | Advantages and contributions

Advantages and contributions

01 One shared world supports controlled policy comparisons

02 Recursive scoring uses the same role vector at every hierarchy level

03 Explicit lease states make assignment behavior inspectable

04 Routed traffic and energy expose the cost of coordination

05 Stored per-seed outputs support further analysis

simulation/; patent-study/run_patent_study.py

30 / 33

Speaking guide

Frame contributions at the project level. The work integrates deterministic topology, a fixed hierarchy, capability calculations, state machines, policy alternatives, failures, solar energy, and reporting into a small modular simulator. It provides more explanatory value than a score-only demonstration because control exchanges and data routing affect batteries.

Its reproducible artifacts are useful for investigating tradeoffs. Avoid claims of patentability, novel scientific priority, or deployment scalability: none follows from this repository alone. The next slide states the boundaries that evaluators are likely to challenge.

Implementation evidence: simulation/; patent-study/run_patent_study.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 31 | Limitations

Limitations

01 Role labels model availability, without anomaly detection or discovery payload algorithms

02 Shared-memory coordination does not simulate consensus, message delay, or split-brain

03 No MAC contention, retransmission, sensing / CPU energy, or calibrated radio units

04 Constant solar rates omit daylight and weather variation

05 Mesa version drift breaks startup, and SciPy is undeclared

simulation/entities.py; simulation/protocols.py; simulation/metrics.py; requirements.txt

31 / 33

Speaking guide

Add several engineering subtleties if asked. Container representation is first available descendant, and initial degree centrality stays fixed. Role availability checks active lease, holder availability, and baseline controller availability, but does not test every application path. Broadcast snapshots default to immediate shared state because interval zero bypasses stale knowledge. Modeled energy counters can exceed actual removed battery energy near zero.

The stored study has only one topology-size family and does not prove asymptotic scalability. Missing dependency pinning and an undeclared SciPy requirement reduce environment reproducibility. Fresh verification: Mesa 3.5.1 raises AttributeError when runner.py assigns scenario. Mesa 3.3.1 executes both smoke simulations. This is a concrete compatibility problem caused by the open mesa>=3 requirement, not just a hypothetical risk.

Implementation evidence: simulation/entities.py; simulation/protocols.py; simulation/metrics.py; requirements.txt

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 32 | Future enhancements

Future enhancements

01 Implement real role services with measurable task correctness

02 Add explicit message queues, delay, partition behavior, and stale-epoch rejection

03 Measure end-to-end root delivery with a fixed offered-load model

04 Calibrate radio and battery costs and introduce time-varying solar profiles

05 Pin dependencies and expand matched scaling and ablation experiments

Architectural recommendations based on simulation/ and study design

32 / 33

Speaking guide

These are proposals, not existing features. A natural first extension is a service interface per role, separating assignment from service execution and defining useful work. Next introduce asynchronous messages and epoch validation at receivers, then test conflicting holders and network partitions.

Track original sensor items through aggregates so root receipt can be measured. Add a realistic energy budget with idle, sensing, and CPU costs. For evaluation, isolate cost awareness, hysteresis, leases, and the gate rather than treating them as one bundle. Test more seeds, node counts, and parameter sensitivities before broad claims.

Implementation evidence: Architectural recommendations based on simulation/ and study design

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 33 | Conclusion and references

Conclusion and references

01 Implemented: hierarchical capability, role leases, routing, recovery, and solar extensions

02 Demonstrated: repeatable comparisons with stored 480-run evidence

03 Main finding: harvest sustains service in this model, while gate benefits remain unproven

04 Evidence: source code, regression tests, per-run metrics, and paired comparisons

Project.md; SIMULATION_GUIDE.md; simulation/; tests/; patent-study/results_20000/

33 / 33

Speaking guide

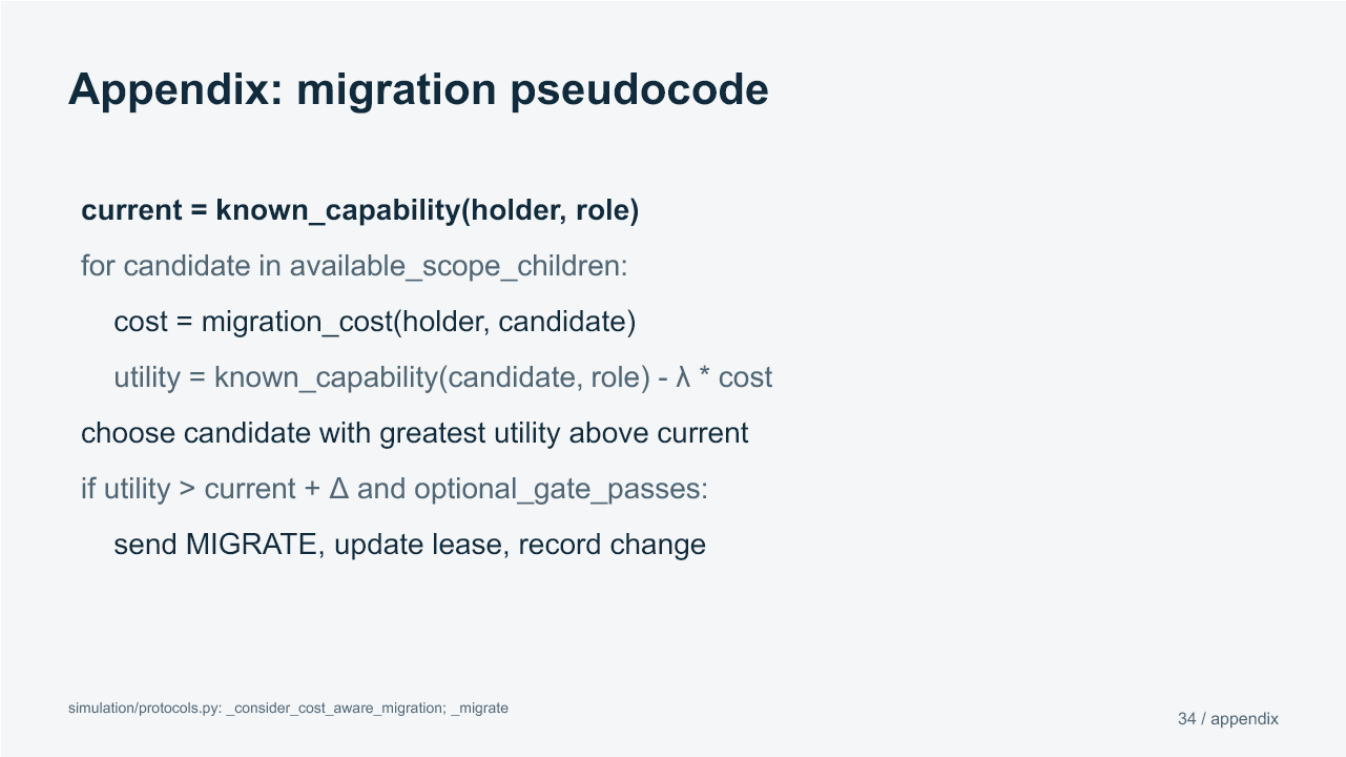
Close by returning to the research question: when is moving a role worth its communication cost? The project gives a reproducible simulation environment to explore that question and makes its assumptions inspectable. The strongest evidence concerns this configured model. Primary internal references are Project.md, FINAL_SPRINT_PLAN.md, SIMULATION_GUIDE.md, nodes.py, simulation modules, tests, and patent-study/results_20000.

The stored final report supplies interpretation but the CSVs support the numbers. Library reference names are Mesa, transitions, NumPy, Pandas, Matplotlib, and SciPy. No external papers were supplied or used as evidence. Transition to the appendix for deeper code and viva discussion.

Implementation evidence: Project.md; SIMULATION_GUIDE.md; simulation/; tests/; patent-study/results_20000/

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 34 | Appendix: migration pseudocode



Speaking guide

This is a faithful summary of _consider_cost_aware_migration rather than a direct code quotation. Cooldown can return before the loop. Infinite costs skip unreachable candidates. Adaptive hysteresis adds .15 times one minus mean node energy.

The solar gate runs only if a candidate already clears utility and hysteresis. The actual migration control exchange can still fail, in which case the lease does not move. Emergency replacement follows _assign instead of this loop. Distinguish the current capability baseline from candidate utility: the current holder has no new switching penalty.

Implementation evidence: simulation/protocols.py: _consider_cost_aware_migration; _migrate

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 35 | Appendix: complexity and data structures

Appendix: complexity and data structures

Operation / storage	Code-derived bound or structure
Topology distance construction	$O(N^2)$ pair comparisons
Capability refresh	$O(R \times \Sigma\, k \log k)$ with child sorting
Uncached single-source routing	$O((N+E) \log N + E \log N + \Sigma$ path lengths$)$
Migration candidate scan	$O(K)$ plus route lookup / construction
Lease inventory	$O(R \times$ scopes$)$, dict of tuple keys
All-source explicit route cache	Up to $O(N^3)$ stored path-node entries

simulation/topology.py; simulation/capability.py; simulation/communications.py; simulation/protocols.py

35 / appendix

Speaking guide

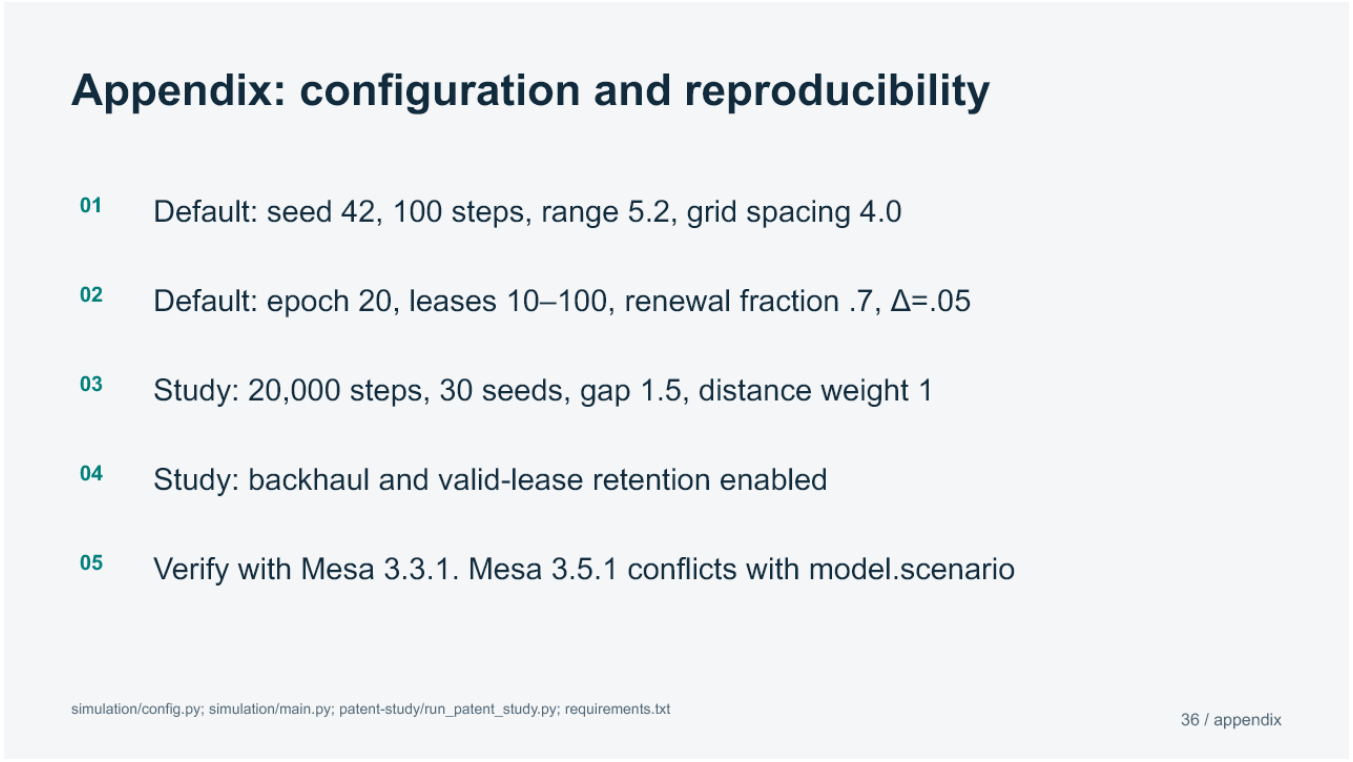
These are code-derived asymptotic interpretations, not measured benchmarks. Let N be physical nodes, E usable edges, K candidates for one role/scope, and R=4 roles. Topology compares node pairs. Aggregation sorts child IDs and performs weighted sums.

Each uncached source builds Dijkstra routes, then materializes each destination path. In a chain the sum of path lengths is quadratic. Caching complete routes for all sources can reach cubic path-node storage in a worst-case graph. Proposed migration scans K candidates and can trigger routes from representatives. This explains why scale tests matter even though default N is only 100.

Implementation evidence: simulation/topology.py; simulation/capability.py; simulation/communications.py; simulation/protocols.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 36 | Appendix: configuration and reproducibility



Speaking guide

Keep default and study configurations separate in your explanation. The basic CLI defaults to five policies and four scenarios, while the study runner defines four particular arms. The initial topology and node attributes are seed-stable and solar randomness uses a separate offset seed.

Persisting a run manifest with exact dependency versions and source hash would strengthen reproducibility. The saved NPZ and per-seed CSV files permit inspection without rerunning all 480 long experiments. The code permits configurable broadcasts, cooldown, affected-scope checks, batching, and detection delay, but the study does not enable every option.

Implementation evidence: simulation/config.py; simulation/main.py; patent-study/run_patent_study.py; requirements.txt

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 37 | Viva preparation 1 of 6

Viva preparation 1 of 6

Why separate logical hierarchy from physical mesh?

Membership defines role candidates and aggregation. Physical adjacency determines whether their representatives can actually communicate.

How many assignments does the default world require?

Four roles across ten subnet, three network, and one internetwork scopes: 4 × 14 = 56.

Does anomaly_detection execute a detection model?

No. It is a role label scored from energy and health. No anomaly classifier or sensor-payload detector is implemented.

How are node scores calculated?

ROLE_WEIGHTS defines normalized weighted sums. Gateway uses .40 energy, .35 link quality, and .25 initial degree centrality.

simulation/entities.py; simulation/topology.py; simulation/config.py; ProtocolEngine.assignment_keys; nodes.py; simulation/runner.py; nodes.py

37 / appendix

Speaking guide

Use these answers as starting points and point to the named implementation when challenged. Explain assumptions explicitly.

If an evaluator asks about measured runtime, radio calibration, deployment security, or statistical coverage beyond these artifacts, state that the repository provides no such evidence. The answers distinguish assignment policy from application service execution and avoid interpreting simulated counters as field measurements.

Implementation evidence: simulation/entities.py; simulation/topology.py; simulation/config.py; ProtocolEngine.assignment_keys; nodes.py; simulation/runner.py; nodes.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 38 | Viva preparation 2 of 6

Viva preparation 2 of 6

What happens to a failed node during aggregation?

Its capability becomes zero but its child weight remains in the denominator, reducing the parent aggregate.

Is the root aggregate the global average node score?

Not necessarily. Default aggregation averages immediate children at each level, and the networks have unequal descendant counts.

How is a container represented on the physical mesh?

The first available descendant node in its stored order represents it. This is an abstraction rather than optimized representative election.

What prevents optional migration oscillation?

Candidate capability minus weighted migration cost must strictly exceed current capability plus hysteresis. Cooldown and adaptive hysteresis are optional.

nodes.py; simulation/capability.py; simulation/capability.py; SimulationConfig.subnet_counts; EntityHierarchy.representative_node_id; ProtocolEng...

38 / appendix

Speaking guide

Use these answers as starting points and point to the named implementation when challenged. Explain assumptions explicitly.

If an evaluator asks about measured runtime, radio calibration, deployment security, or statistical coverage beyond these artifacts, state that the repository provides no such evidence. The answers distinguish assignment policy from application service execution and avoid interpreting simulated counters as field measurements.

Implementation evidence: nodes.py; simulation/capability.py; simulation/capability.py; SimulationConfig.subnet_counts; EntityHierarchy.representative_node_id; ProtocolEngine._consider_cost_aware_migration

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 39 | Viva preparation 3 of 6

Viva preparation 3 of 6

How is migration cost different from charged energy?

The score cost combines estimated messages, transfer, route-based energy, and disruption. Actual router counters charge the modeled exchanges that execute.

How do lease renewal and epoch retention interact?

Renewal updates lease timing at its configured fraction. Without retention, epoch changes regrant leases, potentially occurring before long leases renew.

Does epoch fencing solve distributed split-brain?

No. Epoch labels and invalidation operate in one process. Independent receivers, delayed messages, and stale-epoch rejection are not simulated.

Why does routing sometimes choose more hops?

Dijkstra minimizes quality-weighted cost, so two reliable edges can cost less than one poor edge.

calculate_migration_cost; ProtocolEngine._migrate; ProtocolEngine._renew_if_due; _advance_epoch; simulation/protocols.py; simulation/leases.py; N...

39 / appendix

Speaking guide

Use these answers as starting points and point to the named implementation when challenged. Explain assumptions explicitly.

If an evaluator asks about measured runtime, radio calibration, deployment security, or statistical coverage beyond these artifacts, state that the repository provides no such evidence. The answers distinguish assignment policy from application service execution and avoid interpreting simulated counters as field measurements.

Implementation evidence: calculate_migration_cost; ProtocolEngine._migrate; ProtocolEngine._renew_if_due; _advance_epoch; simulation/protocols.py; simulation/leases.py; NetworkRouter._current_graph; _routes_from

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 40 | Viva preparation 4 of 6

Viva preparation 4 of 6

When are cached routes invalidated?

Failure, recovery, link degradation, and depletion invalidate usable graph and source-route caches.

What is the routing complexity caveat?

Heap-based shortest paths are followed by explicit reconstruction of every destination path. Path storage can dominate textbook distance-only bounds.

How is solar energy modeled?

A constant seeded per-node rate charges batteries up to capacity. Overflow is curtailed, and depleted nodes rejoin after reaching reserve.

Can the solar gate block failure recovery?

No. The gate restricts optional migrations. Assignment after holder failure takes the recovery path instead.

simulation/runner.py; NetworkRouter.invalidate_graph; NetworkRouter._routes_from; RoleSimulationModel._harvest_solar_energy; ProtocolEngine.step;...

40 / appendix

Speaking guide

Use these answers as starting points and point to the named implementation when challenged. Explain assumptions explicitly.

If an evaluator asks about measured runtime, radio calibration, deployment security, or statistical coverage beyond these artifacts, state that the repository provides no such evidence. The answers distinguish assignment policy from application service execution and avoid interpreting simulated counters as field measurements.

Implementation evidence: simulation/runner.py; NetworkRouter.invalidate_graph; NetworkRouter._routes_from; RoleSimulationModel._harvest_solar_energy; ProtocolEngine.step; _solar_migration_is_worthwhile

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 41 | Viva preparation 5 of 6

Viva preparation 5 of 6

What does CSC measure?

The fraction of required role-time intervals whose lease and holder satisfy role_available. It excludes time zero and does not prove service-task execution.

Is sensor delivery measured at the root?

No. It counts successful sensor items reaching subnet gateways. Optional aggregate backhaul has separate counters.

Why can high residual energy coexist with poor service?

A failed controller or unavailable gateway can suppress traffic and leave nodes alive but unable to provide modeled service.

What does right-censored lifetime mean?

The threshold did not cross during the observation horizon. The horizon is a bound on observation, not a measured failure time.

MetricsRecorder.record; ProtocolEngine.role_available; RoleSimulationModel._route_sensor_traffic; simulation/runner.py; stored per_run_metrics.cs...

41 / appendix

Speaking guide

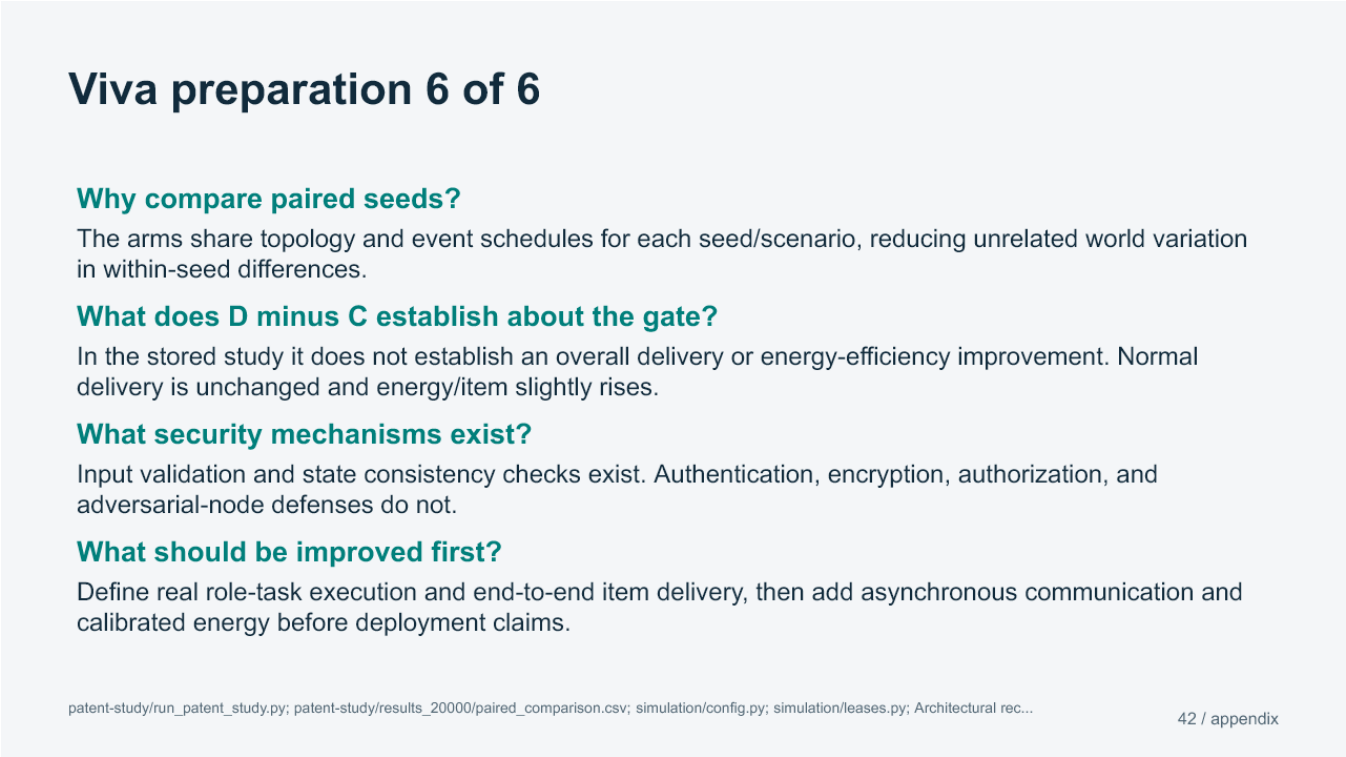
Use these answers as starting points and point to the named implementation when challenged. Explain assumptions explicitly.

If an evaluator asks about measured runtime, radio calibration, deployment security, or statistical coverage beyond these artifacts, state that the repository provides no such evidence. The answers distinguish assignment policy from application service execution and avoid interpreting simulated counters as field measurements.

Implementation evidence: MetricsRecorder.record; ProtocolEngine.role_available; RoleSimulationModel._route_sensor_traffic; simulation/runner.py; stored per_run_metrics.csv; simulation/metrics.py; patent-study/analyze_patent_study.py

Panel prompt: Explain the tradeoff and implementation boundary on this slide.

Slide 42 | Viva preparation 6 of 6



Speaking guide

Use these answers as starting points and point to the named implementation when challenged. Explain assumptions explicitly.

If an evaluator asks about measured runtime, radio calibration, deployment security, or statistical coverage beyond these artifacts, state that the repository provides no such evidence. The answers distinguish assignment policy from application service execution and avoid interpreting simulated counters as field measurements.

Implementation evidence: patent-study/run_patent_study.py; patent-study/results_20000/paired_comparison.csv; simulation/config.py; simulation/leases.py; Architectural recommendation based on implementation

Panel prompt: Explain the tradeoff and implementation boundary on this slide.